

Heaven's Light is our Guide

Rajshahi University of Engineering & Technology

Department of Computer Science & Engineering



Lab Report

Course Code : CSE 1204
Course Name : Object Oriented Programming Sessional
Experiment No : 01
Experiment's Name : Based on class & objects

Submitted By : Name : Mst. Sadia Akter Roll : 2403033 Section : A Session : 2024-2025	Submitted To : Dr. Md. Shahid Uz Zaman Professor Shyla Afroge Associate Professor Department of CSE , RUET
--	---

Date of Submission :

Code:

```
#include <iostream>
using namespace std;
class Account
{
public:
    string name;
    int acn;
    int balance;

    Account()
    {
    }

    void Open_account(string n, int a, int b)
    {
        name = n;
        acn = a;
        balance = b;
    }

    void Delete()
    {
        if (balance > 0)
            cout << "Account can not be deleted. Withdraw your money first ;)" << endl;
        else
        {
            name = " ";
            acn = 0;
            balance = 0;
            cout << "Account deleted successfully" << endl;
        }
    }

    void deposit(int b)
    {
        int old = balance;
        balance = b;
        old += balance;
        balance = old;
    }

    void withdraw(int b)
    {
        int old = balance;
        balance = b;
```

```

        if (b > old)
            cout << "Insufficient balance " << endl;
        else
        {
            old -= b;
            balance = old;
        }
    }
}

void Getdata()
{
    cout << "Account number : " << acn << endl;
    cout << "Name          : " << name << endl;
    cout << "balance        : " << balance << " Taka " << endl;
    cout << ".....\n";
}

};

void print()
{
    cout << "***** Main Menu ***** \n";
    cout << "1. Open New Account " << endl;
    cout << "2. Close Old Account" << endl;
    cout << "3. Deposit money" << endl;
    cout << "4. Withdraw money" << endl;
    cout << "5. Check balance" << endl;
    cout << "6. Exit" << endl;
    cout << "    Enter your option (1-6): " << endl;
}

int main()
{
    int choice;
    int index = 0;
    string Name;
    int Acn;
    int Bal;
    int temp_Acn, temp_balance;

    Account b[10000];

    while (1)
    {
        print();
        cin >> choice;

        switch (choice)
        {
            case 1:
                cout << "Enter your Name: ";
                cin >> Name;

```

```

    cout << "Enter your balance: ";
    cin >> Bal;
    Acn = 1000 + index + 1;

    b[Acn].Open_account(Name, Acn, Bal);
    b[Acn].Getdata();
    index++;
    break;
case 3:
    cout << "Enter your account number: ";
    cin >> temp_Acn;
    if (temp_Acn < (index - 1) && temp_Acn > 1000)
    {
        cout << "Enter your deposite balance: ";
        cin >> temp_balance;
        b[temp_Acn].deposit(temp_balance);
        b[temp_Acn].Getdata();
        cout << "Success" << endl;
    }
    else
        cout << "Invalid account" << endl;

    break;
case 4:
    cout << "Enter your account number: ";
    cin >> temp_Acn;
    if (temp_Acn < (index - 1) && temp_Acn > 1000)
    {
        cout << "Enter your withdrawal balance: ";
        cin >> temp_balance;
        b[temp_Acn].withdraw(temp_balance);
        b[temp_Acn].Getdata();
        cout << "Success" << endl;
    }
    else
        cout << "Invalid account\n";

    break;
case 2:
    cout << "Enter your account number: ";
    cin >> temp_Acn;
    if (temp_Acn < (index - 1) && temp_Acn > 1000)
    {
        b[temp_Acn].Delete();
    }
    else
        cout << "Invalid account" << endl;

```

```

        break;
    case 5:
        cout << "Enter your account number: ";
        cin >> temp_Acn;
        if (temp_Acn < (index - 1) && temp_Acn > 1000)
        {
            cout << "Your current balance: " << b[temp_Acn].balance << " Taka" <<
endl;
        }
        else
            cout << "Invalid account" << endl;

        break;
    case 6:
        cout << "Thank you" << endl;
        return 0;
        break;

    default:
        break;
}
}
}

```

Output Snippet

```

1. Open New Account
2. Close Old Account
3. Deposit money
4. Withdraw money
5. Check balance
6. Exit
    Enter your option (1-6):

```

```

1
Enter your Name: Ragib
Enter your balance: 5000
Account number : 1001
Name           : Ragib
balance        : 5000 Taka
.....

```

```
Enter your option (1-6): 3
Enter your account number: 1001
Enter your deposit balance: 6000
Account number : 1001
Name           : Ragib
Balance        : 11000 Taka
.....
Success
```

```
Enter your option (1-6): 4
Enter your account number: 1001
Enter your withdrawal balance: 3000
Account number : 1001
Name           : Ragib
Balance        : 8000 Taka
.....
```

```
Enter your option (1-6): 2
Enter your account number: 1001
Account can not be deleted. Withdraw your money first ;)
```

```
Enter your option (1-6): 5
Enter your account number: 1001
Your current balance: 8000 Taka
```

Heaven's Light is our Guide

Rajshahi University of Engineering & Technology

Department of Computer Science & Engineering



Lab Report

Course Code : CSE 1204
Course Name : Object Oriented Programming Sessional
Experiment No : 02
Experiment's Name : Constructor, const & static

Submitted By : Name : Mst. Sadia Akter Roll : 2403033 Section : A Session : 2024-2025	Submitted To : Dr. Md. Shahid Uz Zaman Professor Shyla Afroge Associate Professor Department of CSE , RUET
--	---

Date of Submission :

Module 2 [Constructor, const & static] (for Week 2, Based on CO1)

1. Problem Statement: Use the following class Test with the data members and methods

```
class Test{ private:
Data Member x; Data Member y; Data Member z; public:
//write methods

};
```

Code:

```
#include<iostream>
using namespace std;
class Test
{ private:
    int x;
    int y;
    static int z;
public:
    Test()
    { x=0;
      y=0;
      z++; }
    Test(int a, int b)
    { x=a; y=b;
      z++; }
    Test(const Test &obj)
    { x = obj.x;
      y = obj.y;
      z++;
    }
    void setxy(int a, int b)
    {
        x=a;
        y=b;
    }
    void display() const
    {cout << x << y << z << endl;
    }
    int getx()
    {return x;
    }
    int gety()
    {
        return y;
    }
    ~Test()
    {
        cout << "Destroyed" << endl;
    }
};
```

```
int Test::z=0;
int main()
{
    int y[10];
    Test t1;
    Test t2(5,4);
    Test t3(t2);
    Test t4,t5;

    t4.setxy(10,20);
    t5.setxy(25,14);
    int sumx =
    t1.getx()+t2.getx()+t3.getx()+t4.getx()+t5.getx();
    y[0]=t1.gety();
    y[1]=t2.gety();
    y[2]=t3.gety();
    y[3]=t4.gety();
    y[4]=t5.gety();
    int max = y[0];
    for(int i=0; i<5; i++)
    {
        max=(max>y[i]?max:y[i]);
    }
    cout << "Sum of X: " << sumx << endl;
    cout << "Largest Y: " << max << endl;
    return 0;
}
```

Output:

2. **Problem Statement** : Write a class A with two data member **a** and **b**. Now set the values of **a** and **b** of an object **obj1** by a setter method **SetData()** and copy the values of **a** and **b** of **obj1** to another object **obj2** by using copy constructor.

```
#include<iostream>
using namespace std;
class A
{
    int a;
    int b;
public:
    A()
    {

    }

    void setdata(int x, int y)
    {
        a=x;
        b=y;
    }
    A(const A &obj)
    {
        a=obj.a;
        b=obj.b;
    }
    void display()
    {
        cout << a << " " << b << endl;
    }
};
int main()
{
    A obj1;
    obj1.setdata(13,34);
    A obj2(obj1);
    cout << "Object 1: " ; obj1.display();
    cout << "Object 2: " ; obj2.display();

    return 0;
}
```

Output:

```
+ prblm2.cpp -o prblm2 } ; if ($?)
Object 1: 13 34
Object 2: 13 34
```

Heaven's Light is our Guide

Rajshahi University of Engineering & Technology

Department of Computer Science & Engineering



Lab Report

Course Code : CSE 1204
Course Name : Object Oriented Programming Sessional
Experiment No : 03
Experiment's Name : Inheritance

Submitted By : Name : Mst. Sadia Akter Roll : 2403033 Section : A Session : 2024-2025	Submitted To : Dr. Md. Shahid Uz Zaman Professor Shyla Afroge Associate Professor Department of CSE , RUET
--	---

Date of Submission :

Module 3 [Inheritance]: (for Week 4 [29-3/11-12/2025])

Problem Statement: You have to create an inheritance among **Father-->Son -->GrandSon** class. The **father** class has the following data members

Code:

```
#include<bits/stdc++.h>
using namespace std; class
Father {      private: int
money=100;      protected:
int gold=10;      public:
int land=500;

        /*/ Father()          {
money=100;                      gold=10;
land=500;                      cout <<
"created" << endl;
        }                      int
getmoney()                      {
return money;
        }*/ };
class Son : public Father
{      public:
Son()
        {
        }
void display()
        {
        //cout << getmoney() << endl;
cout << gold << endl;          cout <<
land << endl;
        } };
class Grandson : public Son
{      public:
Grandson()          {}
void display()
        {
        //cout << getmoney() << endl;
cout << gold << endl;          cout <<
land << endl;          }
        //int return sum
};
int main()
{      Son
s;
        s.display();

        Grandson g;
        g.display();
return 0;
```

```
}
```

Topic 2 [Types of Inheritance]: Learn and Test different types of inheritance in C++. In each inheritance draw the class diagram with class chain and try to access the data members of bases classes from child classes.

i) Single inheritance

Code:

```
#include <iostream> using
namespace std;
class A {
private:
int x;
protected:
int y;
public:
int z;
A()
{
}
A(int a, int b, int c)
{
x=a;
y=b;
z=c; }
}; class B : public
A { public:
void displayFromA(class A &obj) { cout
<< "Accessing from class B:" << endl; cout
<< "y (protected): " << obj.y << endl; cout
<< "z (public): " << obj.z << endl;
}
B()
{

}
}; int
main() {
A obj1(10,20,30); B
obj2;
obj2.displayFromA(obj1);
return 0; }
```

Multi-level inheritance

```

using namespace std;
class A{ private:
int x=10; protected:
int y=11; public:
int z=12;
int getx() { return x;
} }; class B:public A{ };
class C:private B{ public:
int gety() { return y;
} int getz() { return
z; } int getpvt() {
getx(); } }; int main(){ C
c; cout << c.getpvt() <<
endl; cout << c.gety() <<
endl; cout << c.getz()<<
endl; return 0;
}

```

Multiple inheritance

Code:

```

#include<bits/stdc++.h>
using namespace std;
class A{ private: int
x=1; protected: int y=2;
public: int z=3; int
getx() { return x; }
}; class B{ private: int
p=11; protected: int
q=22; public: int r=33;
int getp() { return
p; } }; class C:public
A,public B
{ public:
int gety() {
return y; }
int getq() {
return q;
}

}; int main(){ C c; cout << " x "
<<c.getx() << endl; cout << " y "
<<c.gety() << endl; cout << " z "
<< c.z<< endl;
cout << " p " << c.getp()<< endl;
cout << " q " << c.getq() << endl;
}

```

```
cout << " r " << c.r<< endl;
return 0; }
```

Heirarchical inheritance

Code:

```
#include <bits/stdc++.h>
using namespace std;
class A { private:
int x; protected:
int y; public:      int
z;      A() {      x
= 7;      y = 8;
z = 9;
      }      int
getX() {
return x;
      }
}; class B : public
A { public:
      void showB() {      cout << "B: x = " << getX() << ", y = " << y << ",
z = " << z << endl;
      }
}; class C : public
A { public:
      void showC() {      cout << "C: x = " << getX() << ", y = " << y << ",
z = " << z << endl;
      }
};
int main() {
B b;      C
c;
      b.showB();
      c.showC();
return 0;
}
```

Hybrid (Diamond) inheritance [virtual class]

Code

```
#include <bits/stdc++.h> using
namespace std;
class A {
private:
```

```

int x;
protected:
int y; public:
int z;      A()
{
    x =
11;        y =
22;        z =
33;
    }      int
getX() {
return x;
    }
}; class B : virtual public
A {
}; class C : virtual public
A {
}; class D : public B, public
C { public:
    void show() {          cout << "x = "
<< getX() << endl;          cout << "y = "
<< y << endl;          cout << "z = " << z
<< endl;
    }
};
int main() {
    D d;
    d.show();
return 0; }

```

Topic 3 [Constructor & Destructor in inheritance]: Write the constructors & destructors for different types of inheritance are given as follows. Also follow and write the sequence of their execution.

i) Single inheritance

<pre> class A{ private: int ax; public: //write constructor to initialize ax //Write destructor } </pre>	<pre> class B:public A{ private: int bx; public: //write constructor to initialize bx //Write method to sum ax and bx //Write destructor } </pre>	<pre> int main(){ B b; //call methods of class B return 0; } </pre>
--	---	---

```

#include <bits/stdc++.h> using
namespace std;
class A { protected:    int ax;
public:    A() {          ax = 10;
cout << "A constructor\n";
    }
~A() {
    cout << "A destructor\n";
    }
}; class B : public A { private:
int bx; public:    B() {
bx = 20;          cout << "B
constructor\n";
    }    void sum() {          cout << "Sum
= " << ax + bx << endl;
    }
~B() {
    cout << "B destructor\n";
    }
}; int
main() {
    B b;
    b.sum();
return 0;
}

```

ii) Multi-level inheritance

```

#include <bits/stdc++.h>
using namespace std;
class A {
protected:
int ax; public:
A() {          ax
= 10;
    cout << "A constructor\n";
    }    ~A() {          cout <<
"A destructor\n";
    }
}; class B : public
A { protected:
int bx; public:
B() {          bx =
20;
    cout << "B constructor\n";
    }    ~B() {          cout << "B destructor\n";

```



```

    }
}; class C : public
B { private:    int
cx; public:    C()
{
    cx = 30;
    cout << "C constructor\n";
}    void
sum() {
    cout << "Sum = " << ax + bx + cx << endl;
}    ~C() {        cout <<
"C destructor\n";
}
}; int
main() {
    C c;
    c.sum();
    return 0;
}

```

Multiple inheritance

```

#include <bits/stdc++.h> using
namespace std; class A {
protected:    int ax; public:
A() {        ax = 10;
cout << "A constructor\n";
}    ~A() {        cout <<
"A destructor\n";
}
}; class B { protected:    int
bx; public:    B() {        bx =
20;        cout << "B
constructor\n";
}    ~B() {        cout <<
"B destructor\n";
}
}; class C : public A, public
B { private:
    int cx;
public:    C()
{
    cx =
30;
    cout << "C constructor\n";
}    void sum() {        cout << "Sum = "
<< ax + bx + cx << endl;
}    ~C() {        cout <<
"C destructor\n";
}
}

```

```
}; int
main() {
    C c;
    c.sum();
    return 0; }
```

Heirarchical inheritance

```
#include <bits/stdc++.h> using
namespace std;
class A { protected:    int ax;
public:    A() {          ax = 10;
cout << "A constructor\n"; }
~A() {          cout << "A
destructor\n";
    }
}; class B : public
A { private:    int
bx; public:    B()
{          bx = 20;
    cout << "B constructor\n";
    } ~B() {          cout <<
"B destructor\n";
    }
}; class C : public
A { private:    int
cx; public:    C()
{          cx = 30;
    cout << "C constructor\n";
    } void sum() {          cout << "Sum
= " << ax + cx << endl;
    } ~C() {          cout <<
"C destructor\n";
    }
}; int
main() {
    B b;    C
    c;
    c.sum();
    return 0; }
```

Hybrid (Diamond) inheritance [virtual class]

```
#include <bits/stdc++.h> using
namespace std; class A {
```

```

protected:    int ax; public:
A() {        ax = 10;
cout << "A constructor\n";
    }    ~A() {        cout <<
"A destructor\n";
    }
}; class B : virtual public A {
protected:    int bx; public:
B() {        bx = 20;
cout << "B constructor\n";
    }    ~B() {        cout <<
"B destructor\n";
    }
}; class C : virtual public
A { protected:    int cx;
public:
    C() {
cx = 30;
        cout << "C constructor\n";
    }    ~C() {        cout <<
"C destructor\n";
    }
}; class D : public B, public
C { private:    int dx;
public:    D() {        dx =
40;
        cout << "D constructor\n";
    }    void sum() {        cout << "Sum = " << ax
+ bx + cx + dx << endl;
    }    ~D() {        cout <<
"D destructor\n";
    }
}; int
main() {
    D d;
    d.sum();
return 0;
}

```

Heaven's Light is our Guide

Rajshahi University of Engineering & Technology

Department of Computer Science & Engineering



Lab Report

Course Code : CSE 1204
Course Name : Object Oriented Programming Sessional
Experiment No : 04
Experiment's Name : Method Or Function Overriding

Submitted By :	Submitted To :
Name : Mst. Sadia Akter	Dr. Md. Shahid Uz Zaman
Roll : 2403033	Professor
Section : A	Shyla Afroge
Session : 2024-2025	Associate Professor
	Department of CSE ,
	RUET

Topic 1[Method/Function Overloading]

Problem Statement: Define a class Test where overload a method Sum() to sum numbers sent from main() function.

Solution:

```
#include<bits/stdc++.h> using namespace std; class Test { public:

    int Sum(int t)
    {
        return t;
    }    int Sum(int a, int b)

    {
        return a+b;
    }    double Sum(double a, int b)

    {
        return a+b;
    }    double Sum(int a, double b)

    {
        return a+b;
    }    double Sum(double a, double b)

    {
        return a+b;
    }
}; int
main() {

    Test t;    cout << t.Sum(10) << endl;    cout
<< t.Sum(10,20) << endl;    cout << t.Sum(5.7,20) <<
endl;    cout << t.Sum(10,2.6) << endl;    cout <<
t.Sum(10.5,20.7) << endl;    return 0;

}
```

Output

:

```

adding }
10
30
25.7
12.6
31.2

```

Topic 2 [Binary Operator Overloading]: Suppose in a AC circuit, there are 3 impedances $z_1=3+j4$, $z_2=4-j3$ and $z_3=j6$ are connected in parallel. Now find the current in the circuit if input voltage is $100+j50$. Implement operator overloading concept for your calculation. Use class Circuit and initialize the impedance values (real & img) by a constructor.

Solution:

```

#include <iostream> using
namespace std;

class Circuit {
private:
    double real;    double img;

public:
    Circuit(double r = 0, double i = 0) {        real = r;
img = i;
    }
    Circuit operator+(Circuit c) {        return Circuit(real + c.real, img +
c.img);
    }
    Circuit operator/(Circuit c) {        double denom = c.real * c.real + c.img *
c.img;        double r = (real * c.real + img * c.img) / denom;        double i =
(img * c.real - real * c.img) / denom;        return Circuit(r, i);

```

```

    }    void display() {        if (img >= 0)            cout << real << "
+ j" << img << endl;        else            cout << real << " - j" << -img <<
endl;

    }
};    int main() {        Circuit
z1(3, 4);

        Circuit z2(4, -3);
        Circuit z3(0, 6);
        Circuit V(100, 50);
        Circuit one(1, 0);
        Circuit y1 = one / z1;
        Circuit y2 = one / z2;
        Circuit y3 = one / z3;

        Circuit Yeq = y1 + y2 + y3;

        Circuit Zeq = one / Yeq;

        Circuit I = V / Zeq;

        cout << "Equivalent Impedance Z = ";
        Zeq.display();

        cout << "Circuit Current I = ";
        I.display();
    return 0;

}

```

Topic 3 [Unary Operator Overload]

Solution:

```
#include <iostream>
```

```

using namespace std; class
AirConditioner

{   int temperature;
public:

    AirConditioner(int t)
    {   temperature = t;

    }   void operator++()

    {   temperature++;

    }   void operator--()

    {   temperature--;

    }
    void display()
    {   cout << "Current Temperature: " << temperature << endl;

    } }; int
main()

{
    AirConditioner ac(24);
ac.display();   ++ac;   ac.display();
--ac;   ac.display();   return 0;

}

```

Topic 4[Method/Function Overriding]

Solution:

```

#include <iostream> using
namespace std; class A{
public:

    void Print(){   cout << "Inside Print() of class A" << endl;

    } }; class B : public A{
public:

    void Print(){   cout << "Inside Print() of class B" << endl;

```



```

    } }; int
main() {
A a;

    a.Print();
    B b;
    b.Print();
A    a2;  A *p1;  p1 = &a2;  p1->Print();
B    b2;  A *p2;  p2 = &b2;  p2->Print();  return 0;
}

```

Topic 5 [Friend Function]

Problem Statement: Using the following class, write three friend functions

- i) Add(): Assign value to the data member x
- ii) IncX() : Increase the value of x by m
- iii) DecX() : Decrease the value of x by n

Solution:

```

#include <iostream> using namespace std; class Sample

{   int x; public:   Sample()   {   x = 0;   }   friend
void Add(Sample &s, int value);   friend void IncX(Sample &s,
int m);   friend void DecX(Sample &s, int n);   void display()

{   cout << "Value of x: " << x << endl;

} }; void Add(Sample &s, int value)

{

s.x = value;
} void IncX(Sample &s, int m)

{

s.x = s.x + m; } void DecX(Sample
&s, int n)

{

s.x = s.x - n; } int
main()

```

```

{
    Sample obj;    Add(obj, 10);
obj.display();    IncX(obj, 5);
obj.display();    DecX(obj, 3);
obj.display();    return 0;
}

```

Topic 6[Pure Virtual Function]

Problem statement: Modify the class defined in Topic 3 executes the following

```

#include <iostream> using
namespace std;

class A{
public:
virtual
void
Print() =
0;

}; class B : public A{
public:

void Print(){    cout << "Inside Print() of class B" << endl;

}

};

```

Heaven's Light is our Guide

Rajshahi University of Engineering & Technology

Department of Computer Science & Engineering



Lab Report

Course Code : CSE 1204
Course Name : Object Oriented Programming Sessional
Experiment No : 05
Experiment's Name : Advanced Topics

Submitted By :	Submitted To :
Name : Mst. Sadia Akter	Dr. Md. Shahid Uz Zaman
Roll : 2403033	Professor
Section : A	Shyla Afroge
Session : 2024-2025	Associate Professor
	Department of CSE ,
	RUET

Date of Submission :

Module 5 [Advanced Topics]:

Topic 1[Exception Handling]

Problem Statement: In the following input *i* is the index of array *ax[]* . The program prints *ax[i]* . Then write catch block if *i* is out of range of *ax[]* . Write three catch blocks to fulfill the purpose

- i) a catch block receives the value of *i*
- ii) a catch block receives string "Out of Range Error"
- iii) a default catch() if above two catch block doesn't match

Code:

```
#include <iostream>
using namespace std;

int main()
{
    int i;
    int ax[5]={10,20,60,40,30};
    cout<<"Enter index(int):";
    cin>>i;
    try
    {
        if(i>4 || i<0)
        {
            throw(i);
        }
        if(cin.fail())
        {
            throw("notinteger");
        }
        cout<<"ax["<<i<<"]="<<ax[i]<<endl;
    }
    catch(int i)
    {
        cout << "Out of Range" << endl;
    }
    catch(const char *s)
    {
        cout << s << endl;
    }
    catch(...)
    {
        cout << "Exception is exception" << endl;
    }
    return 0;
}
```

Topic 2 [class Template]:

Problem Statement: In the following class data members x and y are integers and the method Sum() adds x and y. However, we need to perform the sum of int+int, int+double, double+int and double+double . To achieve it, change the definition of x,y,setData() and Sum() accordingly.

```
class A{ private:
int x; int y;
public:
void setData(int x,int y){ this->x=x;
this->y=y;
}
int Sum(){ int s; s=x+y; return s;
}
};
int main(){
//write required statements to call SetData() & Sum()
}
```

Code:

```
#include <iostream>
using namespace std;
template <class T1, class T2>
class A
{
    T1 x;
    T2 y;
public:
    void setData(T1 x, T2 y) {
        this-> x=x;
        this ->y =y;
    }
    auto Sum()
    {
        return x + y;
    }
};
int main()
{

    A<int, int> obj1;
    obj1.setData(10, 20);
    cout << "Sum(int, int): " << obj1.Sum() << endl;
```

```

A<int, double> obj2;
obj2.setData(10, 5.5);
cout << "Sum(int, double): " << obj2.Sum() << endl;
A<double, int> obj3;
obj3.setData(3.5, 7);
cout << "Sum(double, int): " << obj3.Sum() << endl;
A<double, double> obj4;
obj4.setData(2.5, 3.5);
cout << "Sum(double, double): " << obj4.Sum() << endl;
return 0;
}

```

Topic 3 [STL:Array class]

Problem statement: Declare a STL array object ax with 6 elements and do the following:

- i) Assign 10,60,30,70,20 to ax using single statement
- ii) Print third element of ax using at() function
- iii) Print first element of ax using front() function
- iv) Print last element of ax using back() function
- v) Fill the elements of ax using fill() function
- vi) Test whether ax is empty or not using empty() function
- vii) Print size of ax
- viii) Print maximum size of ax using max_size() function
- ix) Print address of first element of ax using begin() function
Print address of last element of ax using end()

```

#include <iostream>
#include <bits/stdc++.h>
using namespace std;

int main() {
    array<int, 6> ax;
    ax = {10, 60, 30, 70, 20, 90};
    cout << "Third element: " << ax.at(2) << endl;
    cout << "First element: " << ax.front() << endl;
    cout << "Last element: " << ax.back() << endl;
    ax.fill(5);
    cout << "After fill(): ";
    for (int x : ax) cout << x << " ";
    cout << endl;
    cout << "Is empty: " << (ax.empty() ? "Yes" : "No") << endl;
    cout << "Size: " << ax.size() << endl;
    cout << "Max size: " << ax.max_size() << endl;
    cout << "Address of first element: " << &(*ax.begin()) << endl;
    cout << "Address of last element: " << &(*(ax.end() - 1)) << endl;

    return 0;
}

```

Topic 4[STL: pair class]

Problem statement: Define a pair class object px with int and string elements. Write statements to do the following

- i) Assign 10 to int and "Rajshahi" to px using make_pair() function
- ii) Print int data member by first
- iii) Print string data member by second
- iv) Modify first data member to 20 using get<>() function
- v) Declare another pair bx and assign values to bx and swap it with ax

Code:

```
#include <iostream>
#include <bits/stdc++.h>
using namespace std;

int main() {
    pair<int, string> px;
    px = make_pair(10, "Rajshahi");
    cout << "First: " << px.first << endl;
    cout << "Second: " << px.second << endl;
    get<0>(px) = 20;
    cout << "Modified First: " << px.first << endl;
    pair<int, string> bx;
    bx = make_pair(50, "Dhaka");
    px.swap(bx);
    cout << "After swap, px = (" << px.first << ", " << px.second << ")" << endl;
    cout << "After swap, bx = (" << bx.first << ", " << bx.second << ")" << endl;

    return 0;
}
```

```
g++ stcpair.cpp -o stcpair ; ./stcpair
First: 10
Second: Rajshahi
Modified First: 20
After swap, px = (50, Dhaka)
After swap, bx = (20, Rajshahi)
```

Topic 5 [STL: tuple class]

Problem statement: Define a tuple class object tx with int,string and double elements. Write statements to do the following

- i) Assign <100,"Kamal",3.5> to tx using make_tuple() function
- ii) Print int data member by get() function

- iii) Print string data member by get() function
- iv) Print double data member by get() function
- v) Modify third data member to 3.7 using get<>() function
- vi) Declare another tuple bx and assign values to bx and swap it with ax

Code:

```
#include <iostream>
#include <bits/stdc++.h>
using namespace std;

int main() {
    tuple<int, string, double> tx;
    tx = make_tuple(100, "Kamal", 3.5);
    cout << "Int: " << get<0>(tx) << endl;
    cout << "String: " << get<1>(tx) << endl;
    cout << "Double: " << get<2>(tx) << endl;
    get<2>(tx) = 3.7;
    cout << "Modified Double: " << get<2>(tx) << endl;
    tuple<int, string, double> bx;
    bx = make_tuple(200, "Rahim", 4.2); tx.swap(bx);

    cout << "After swap, tx = ("
        << get<0>(tx) << ", "
        << get<1>(tx) << ", "
        << get<2>(tx) << ")" << endl;

    cout << "After swap, bx = ("
        << get<0>(bx) << ", "
        << get<1>(bx) << ", "
        << get<2>(bx) << ")" << endl;

    return 0;
}
```


Topic 6 [STL: stack class]

Problem statement: Write a program to create and manipulate Stack using stack class and Perform the following operations using the specified method.

- i) use push() method to push data
- ii) use pop() method to pop data
- iii) use top() method to display top element
- iv) use empty() method to check whether stack is empty or not

Code:

```
#include <iostream>
#include <bits/stdc++.h>
using namespace std;

int main() {
    stack<int> st;

    // push()
    st.push(10);
    st.push(20);
    st.push(30);

    // top()
    cout << "Top element: " << st.top() << endl;

    // pop()
    st.pop();
    cout << "After pop, top element: " << st.top() << endl;

    // empty()
    if (st.empty())
        cout << "Stack is empty" << endl;
    else
        cout << "Stack is not empty" << endl;

    return 0;
}
```

Topic 7 [STL: queue class]

Problem statement: Write a program to create and manipulate Queue using queue class. Perform the following operations using the specified method.

- i) use push() method to push data
- ii) use pop() method to pop data
- iii) use front() method to display front element
- iv) use back() method to display rear element
- v) use empty() method to check whether queue is empty or not

Code:

```
#include <iostream>
#include <bits/stdc++.h>
using namespace std;

int main() {
    queue<int> qu;
    qu.push(10);
    qu.push(20);
    qu.push(30);
    cout << "Front element: " << qu.front() << endl;
    cout << "Rear element: " << qu.back() << endl;
    qu.pop();
    cout << "After pop, front element: " << qu.front() << endl;
    if (qu.empty())
        cout << "Queue is empty" << endl;
    else
        cout << "Queue is not empty" << endl;

    return 0;
}
```

Topic 8 [STL: list class]

Problem statement: Write a program to create and manipulate linked list using list class and its following methods to

Code: #include <iostream>
#include <bits/stdc++.h>
#include <iterator>
#include <algorithm>
using namespace std;

```
bool isEven(int x) {
    return x % 2 == 0;
}
```

```
void Display(list<int> li) {
    list<int>::iterator it;
    for (it = li.begin(); it != li.end(); it++) {
```

```

        cout << *it << " ";
    }
    cout << endl;
}

void DisplayRev(list<int> li) {
    list<int>::reverse_iterator it;
    for (it = li.rbegin(); it != li.rend(); it++) {
        cout << *it << " ";
    }
    cout << endl;
}

int main() {
    list<int> li;

    // i) insert 8 integers using push_back()
    li.push_back(10);
    li.push_back(20);
    li.push_back(30);
    li.push_back(40);
    li.push_back(50);
    li.push_back(60);
    li.push_back(70);
    li.push_back(80);

    // ii) insert two using push_front()
    li.push_front(5);
    li.push_front(1);
    cout << "Forward: ";
    Display(li);

    // iv) reverse display
    cout << "Reverse: ";
    DisplayRev(li);

    // v) front()
    cout << "Front element: " << li.front() << endl;

    // vi) back()
    cout << "Back element: " << li.back() << endl;

    // vii) delete back using pop_back()
    li.pop_back();
    cout << "After pop_back: ";
    Display(li);
}

```

```

// viii) delete front using pop_front()
li.pop_front();
cout << "After pop_front: ";
Display(li);

// ix) search an element x using find()
int x = 30;
auto it = find(li.begin(), li.end(), x);
if (it != li.end())
    cout << x << " found" << endl;
else
    cout << x << " not found" << endl;

// x) insert a new element x before existing y
int newVal1 = 25, y1 = 30;
auto it1 = find(li.begin(), li.end(), y1);
if (it1 != li.end()) {
    li.insert(it1, newVal1);
}
cout << "After insert before 30: ";
Display(li);

// xi) insert a new element x after existing y
int newVal2 = 35, y2 = 30;
auto it2 = find(li.begin(), li.end(), y2);
if (it2 != li.end()) {
    ++it2;
    li.insert(it2, newVal2);
}
cout << "After insert after 30: ";
Display(li);

cout << "Count of 30: " << count(li.begin(), li.end(), 30) << endl;
cout << "Count of even elements: " << count_if(li.begin(), li.end(), isEven) <<
endl;

// xiv) delete a particular element x with erase()
auto it3 = find(li.begin(), li.end(), 25);
if (it3 != li.end()) {
    li.erase(it3);
}
cout << "After erase(25): ";
Display(li);

auto start = li.begin();
auto finish = li.begin();
advance(finish, 4);
li.erase(start, finish);

```

```

cout << "After deleting first 4 elements: ";
Display(li);
li.remove(60);
cout << "After remove(60): ";
li.remove_if(isEven);
cout << "After remove_if(isEven): ";
Display(li);
list<int> li2;
li2.push_back(100);
li2.push_back(200);
li2.push_back(300);

li.assign(li2.begin(), li2.end());
cout << "After assign from another list: ";
Display(li);

// xix) assign elements from an array
int arr[] = {7, 8, 9, 10};
li.assign(arr, arr + 4);
cout << "After assign from array: ";
Display(li);

// xx) sort the list
li.push_back(2);
li.push_back(15);
li.sort();
cout << "After sort: ";
Display(li);

// xxi) delete consecutive similar elements using unique()
li.push_back(15);
li.push_back(15);
li.push_back(20);
li.push_back(20);
li.sort();
cout << "Before unique: ";
Display(li);

li.unique();
cout << "After unique: ";
Display(li);

return 0;
}

```

```

After erase(25): 5 10 20 30 35 40 50 60 70
After deleting first 4 elements: 35 40 50 60 70
After remove(60): 35 40 50 70
After remove_if(isEven): 35
After assign from another list: 100 200 300
After assign from array: 7 8 9 10
After sort: 2 7 8 9 10 15
Before unique: 2 7 8 9 10 15 15 15 20 20
After unique: 2 7 8 9 10 15 20 _

```

Heaven's Light is our Guide

Rajshahi University of Engineering & Technology

Department of Computer Science & Engineering



Lab Report

Course Code : CSE 1204
Course Name : Object Oriented Programming Sessional
Experiment No : 06
Experiment's Name : Data Structure with STL

Submitted By : Name : Mst. Sadia Akter Roll : 2403033 Section : A Session : 2024-2025	Submitted To : Dr. Md. Shahid Uz Zaman Professor Shyla Afroge Associate Professor Department of CSE , RUET
--	---

Date of Submission :

Module 06 [Data Structure with STL] (for Week 7)

Topic 1 [STL: stack class]

Problem statement: Write a menu program to create and manipulate Stack using stack class and perform the following operations using the specified method.

- i) use push() method to push data ii) use pop() method to pop data iii) use top() method to display top element
- iv) use empty() method to check whether stack is empty or not

```
** Stack **  
1. Push  
2. Pop  
3. Display()  
4. Exit  
Enter your option:
```

Code:

```
#include<bits/stdc++.h> using  
namespace std;  
int main(){  
stack<int> st;  
int opt, val;  
while(true){          cout<<"\n1.Push\n2.Pop\n3.Top\n4.Check  
Empty\n5.Exit\n";      cin>>opt;  
  
    switch(opt){  
case 1:  
        cout<<"Enter value: ";  
cin>>val;  
st.push(val);          break;  
case  
2:  
        if(!st.empty()){  
cout<<"Popped: "<<st.top()<<endl;  
st.pop();  
        } else cout<<"Stack Empty\n";  
break;  
case  
3:  
        if(!st.empty())  
cout<<"Top: "<<st.top()<<endl;  
else cout<<"Stack Empty\n";          break;  
case 4:  
cout<<(st.empty()?"Empty":"Not Empty")<<endl;  
break;
```

```

                    case
5:
                    return 0;
                }
            }
        }
    }
}

```

Topic 2 [STL: queue class]

Problem statement: Write a menu program to create and manipulate Queue using queue class. Perform the following operations using the specified method.

- i) use push() method to enqueue data ii) use pop() method to dequeue data iii) use front() method to display front element iv) use back() method to display rear element
- v) use empty() method to check whether queue is empty or not

```

** Queue **
1. Enqueue
2. Dequeue
3. Display Front
4. Display Rear
5. Exit
Enter your option:

```

Code:

```

#include<bits/stdc++.h> using
namespace std;
int main(){
queue<int> q;
int opt, val;
while(true){
cout<<"\n1.Enqueue\n2.Dequeue\n3.Front\n4.Rear\n5.Exit\n";
cin>>opt;

switch(opt){
case 1:
        cin>>val;
        q.push(val);
        break;

        case 2:
            if(!q.empty()){
cout<<"Removed: "<<q.front()<<endl;
                q.pop();
            } else cout<<"Queue Empty\n";
            break;

        case
3:

```



```

                if(!q.empty())
cout<<"Front: "<<q.front()<<endl;
break;

                case
4:
                if(!q.empty())
cout<<"Rear: "<<q.back()<<endl;
break;

                case
5:
                return 0;
            }
        }
    }
}

```

Topic 3 [STL: vector class]

Problem statement: Write a menu program to create and manipulate linked list using vector class and use the following methods

i) **push_back()** ii)
 push_front() iii)
 begin() iv) **end()**
 v) **pop_back()** vi)
pop_front() vii)
insert() viii)
erase()

```

** Linked List **
1. Insert 2.
Delete 3.
Update
4. Search
5. Exit
Enter your option:

```

Code:

```

#include<bits/stdc++.h> using
namespace std;
int main(){
vector<int> v;      int
opt, val, pos;
    while(true){
cout<<"\n1.Insert\n2.Delete\n3.Update\n4.Search\n5.Exit\n";
cin>>opt;

switch(opt){
case 1:
            cin>>val;
            v.push_back(val);

break;

```

```

                case
2:
                cin>>pos;
if(pos>=0 && pos<v.size())
                v.erase(v.begin()+pos);
break;

                case
3:
                cin>>pos>>val;
if(pos>=0 && pos<v.size())
v[pos]=val;
                break;

                case
4:
                cin>>val;
                for(int
i=0;i<v.size();i++)
if(v[i]==val)
                cout<<"Found
at "<<i<<endl;
                break;

                case 5:
                return 0;
        }
    }
}

```

Topic 3.1 [STL: vector class]: Add more feature to the menu of Topic 3.

Module 07 [UML+Java Basics]

(for Week 8: 19-23/7/2025)

Topic 1 [UML]

Problem statement: Draw the UML diagram for the following class

Topic 2 [UML: Aggregation]

Problem statement: Create two classes Doctor and Patient to store the information of doctors and patients respectively. Now write codes to create association between these two classes by aggregation. [note: if Patient class is destroyed then Doctor class is still exist]

class Doctor{ //write your code }	class Patient{ //write your code }	int main(){ //write your code }
---	--	---------------------------------------

Code:

```
#include<bits/stdc++.h>
using namespace std; class
Student
{
public:
    str#include<bits/stdc++.h> using
namespace std;
    class
    Patient{
public:
        string name;
        Patient(string n){ name = n; }
}; class
    Doctor{
public:
        Patient* p; // aggregation (reference)

        Doctor(Patient* p){
this->p = p;
        }
        void show(){
            cout<<"Patient: "<<p->name<<endl;
        }
};
int main(){
    Patient p("Ragib");
    Doctor d(&p);

    d.show();
}ing name;
Student(string n)
{
```

```

        name = n;
    }
};

```

Topic 3 [UML: Composition]

Problem statement: Create two classes Doctor and Patient to store the information of doctors and patients respectively. Now write codes to create association between these two classes by composition.

[note: if Doctor class is destroyed then Patient class is also destroyed]

class Doctor{ //write your code }	class Patient{ //write your code }	int main(){ //write your code }
---	--	---------------------------------------

Code:

```

#include<bits/stdc++.h> using
namespace std;
class
Patient{
public:
    string name;
    Patient(string n){ name=n; }
}; class
Doctor{
public:
    Patient p; // composition

    Doctor(string name): p(name){}
    void show(){
cout<<"Patient: "<<p.name<<endl;
    }
};
int main(){    Doctor
d("Ragib");
    d.show(); }

```

Topic 4: [java Programming Basics]

Problem Statement: Write java programs to do the following

i) Display your name and address in two lines ii)

Take two integers as inputs and prints the bigger one

iii) Input a series of 10 numbers into an array ax[] and finds the largest, smallest and average of these numbers. Also write code for searching and sorting. **Code(i):**

```

class A{
    public static void main(String[] args){
System.out.println("Ragib Ishrak");
        System.out.println("Bangladesh");
    }
}

```

```

} Code(ii): import java.util.*; class A{
    public static void main(String[] args){
Scanner sc=new Scanner(System.in);          int
a=sc.nextInt(), b=sc.nextInt();
        System.out.println(Math.max(a,b));
    }
}
Code(iii): import java.util.*; class A{
    public static void main(String[] args){
Scanner sc=new Scanner(System.in);          int
a=sc.nextInt(), b=sc.nextInt();
        System.out.println(Math.max(a,b));
    }
}

```

Topic 5 [java Programming private method]

Problem Statement: Verify that private methods can be used only inside the class but it cannot be used from outside the class.

Code:

```

class A{
    private void show(){
        System.out.println("Private method");
    }
    void call(){
show(); // allowed
    }
    public static void main(String[] args){
A obj=new A();          obj.call();
        // obj.show(); ✗ not allowed
    }
}

```

Topic 6 [java Programming static method/member]

Problem Statement: Verify that static methods can access static method/member but cannot access to non-static method/member.

Code:

```

class A{          static
int x=10;          int
y=20;

    static void show(){
        System.out.println(x); // allowed
        // System.out.println(y); ✗ not allowed
    }
}

```

```
    }    public static void main(String[]  
args){    show();  
    }  
}
```

Topic 7 [java Programming]

Problem Statement: Write a Java Program to initialize radius of a circle using constructor/Setter and calculate area of the circle.

Code:

```
class Circle{  
    double r;  
  
    Circle(double r){  
this.r=r;  
    }  
    double area(){  
return Math.PI*r*r;  
    }  
    public static void main(String[] args){  
        Circle c=new Circle(5);  
        System.out.println("Area="+c.area());  
    }  
}
```

Heaven's Light is our Guide

Rajshahi University of Engineering & Technology

Department of Computer Science & Engineering



Lab Report

Course Code : CSE 1204
Course Name : Object Oriented Programming Sessional
Experiment No : 07
Experiment's Name : Java Basics

Submitted By : Name : Mst. Sadia Akter Roll : 2403033 Section : A Session : 2024-2025	Submitted To : Dr. Md. Shahid Uz Zaman Professor Shyla Afroge Associate Professor Department of CSE , RUET
--	---

Date of Submission :

Module 07 [Java Basics]

Topic 1 [java Programming private method]

Problem Statement: Verify that private methods can be used only inside the class but it cannot be used from outside the class.

Code:

```
class A
{
    private void show()
    {
        System.out.println("Private Method");
    }
    public void display()
    {
        System.out.println("Withen class");
        show();
    }
}

public class Topic1
{
    public static void main(String[] args)
    {
        A a = new A();
        a.display();
        //a.show();
    }
}
```

Topic 2 [java Programming static method/member]

Problem Statement: Verify that static methods can access static method/member but cannot access to non-static method/member.

Code:

```
class A
{
    public static int x = 10;
    int y = 20;
    void display()
    {
        System.out.println("Non static: "+ y);
    }

    public static void show()
    {
        System.out.println("Static method: "+x);
    }
}
```



```

    }
}
public class Topic2
{
    public static void main(String[] args)
    {
        A a = new A();
        a.display();
        a.show();
    }
}

```

Topic 3 [java Programming: Anonymous object]

Problem Statement: Write a Java program that creates a Point object using anonymous object using the following getPoint() method.

Point pt=getPoint();

Code:

```

class Point {
    int x;
    int y;

    Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    void showPoint() {
        System.out.println("X = " + x + ", Y = " + y);
    }
}

public class Mainpoint {

    static Point getPoint() {
        return new Point(5, 10);
    }

    public static void main(String[] args) {
        getPoint().showPoint();
    }
}

```

Topic 4 [java Programming: ArrayList<> object]

Problem Statement: Write a Java program performs the following operations using ArrayList Object

Code:

```
import java.util.*;
public class Topic2
{
    public static void main(String[] args)
    {
        ArrayList<String> ax = new ArrayList<>();
        ax.add("Rose");
        ax.add("Lilie");
        ax.add("Carnation");
        ax.add("Daisie");
        ax.add("Tulip");

        System.out.println("Initial Array: "+ax);

        ax.add(1,"Peonie");
        System.out.println("After Insertion: "+ ax);

        ax.remove("Lilie");
        System.out.println("After Deletion: "+ ax);

        String search = "Tulip";
        if(ax.contains(search))
        {
            System.out.println(search+"found in the list");
        }
        else
        {
            System.out.println(search + "not found");
        }

        Collections.sort(ax);
        System.out.println("Sorted list"+ ax);

        ax.clear();
        System.out.println("After Clearing: "+ ax);
    }
}
```

Topic 5 [inheritance: use of super keyword]

Problem Statement: For the following Java program write

- i) to display x of class A in class B
- ii) statement to call getX() of class A in class B
- iii) to call parameterized constructor of in class B

Code:

```
class A
{
    int x;
    public A()
    {
        x=0;
    }
    public A(int x)
    {
        this.x=x;
    }
    public int getX()
    {
        return(x+10);
    }
}
class B extends A
{
    int x=20;
    public int getX()
    {
        return(x+10);
    }
    public int getXa()
    {
        return super.x;
    }
    public void show()
    {
        System.out.println(super.getX());
    }
    B(int x)
    {
        super(x);
        System.out.println("Super Parameterized Constructor");
    }
}
public class First
{
    public static void main(String[] args)
```

```

        B b = new B(10);
        System.out.println(b.getXa());
        System.out.println(b.getX());
        b.show();
    }
}

```

Topic 6 [interface in Java]

Problem Statement: Write three interfaces in Java namely AI, BI and CI with the following activities:

Code:

```

interface AI
{
    void printA();
}
interface BI
{
    void printB();
}

interface CI
{
    void printC();
}
class A implements AI
{
    public void printA()
    {
        System.out.println("Print A");
    }
}
class B extends A implements BI
{
    public void printB()
    {
        System.out.println("Print B");
    }
}
class C extends B implements CI
{
    public void printC()
    {
        System.out.println("Print C");
    }
}
public class interfacejava
{
    public static void main(String[] args)
    {
        C obj = new C();
        obj.printA();
        obj.printB();
        obj.printC();
    }
}

```

Heaven's Light is our Guide

Rajshahi University of Engineering & Technology

Department of Computer Science & Engineering



Lab Report

Course Code : CSE 1204
Course Name : Object Oriented Programming Sessional
Experiment No : 08
Experiment's Name : Advanced Topics: Java Programming

Submitted By : Name : Mst. Sadia Akter Roll : 2403033 Section : A Session : 2024-2025	Submitted To : Dr. Md. Shahid Uz Zaman Professor Shyla Afroge Associate Professor Department of CSE , RUET
--	---

Date of Submission :

Module 08 [Advanced Topics: Java Programming]: (for Week 11:28-1/3/2026)

Topic 01 [java Programming: Anonymous object]

Problem Statement: Write a Java program that creates a Point object using anonymous object using the following getPoint() method.

Point pt=getPoint();

Code:

```
class Point
{
    int x;
    int y;
    Point(int x, int y)
    {
        this.x=x;
        this.y=y;
    }
    public static Point getPoint(int x, int y)
    {
        return new Point(x, y);
    }
    public void display()
    {
        System.out.println("Point("+x+", "+y+"");
    }
}

public class Topic01 {
    public static void main(String[] args) {
        Point pt = Point.getPoint(5,10);
        pt.display();
    }
}
```

Topic 02 [java Programming: ArrayList<> object]

Problem Statement: Write a Java program performs the following operations using ArrayList Object

Declare a ArrayList object ax to store flowers

Add three flowers: Rose, Lilie, Carnation, Daisie, and Tulip

Insert a new flower Peonie in the second position of ax

Delete the flower Lilie

Find any flower in ax

Sort the list

Delete the list

Code:

```
import java.util.*;
public class Topic2
{
    public static void main(String[] args)
    {
        ArrayList<String> ax = new ArrayList<>();
        ax.add("Rose");
        ax.add("Lilie");
        ax.add("Carnation");
        ax.add("Daisie");
        ax.add("Tulip");

        System.out.println("Initial Array: "+ax);

        ax.add(1,"Peonie");
        System.out.println("After Insertion: "+ ax);

        ax.remove("Lilie");
        System.out.println("After Deletion: "+ ax);

        String search = "Tulip";
        if(ax.contains(search))
        {
            System.out.println(search+"found in the list");
        }
        else
        {
            System.out.println(search + "not found");
        }

        Collections.sort(ax);
        System.out.println("Sorted list"+ ax);

        ax.clear();
        System.out.println("After Clearing: "+ ax);
    }
}
```

Topic 03 [inheritance: use of super keyword]

Problem Statement: For the following Java program write
to display x of class A in class B
statement to call getX() of class A in class B
to call parameterized constructor of in class B

Code:

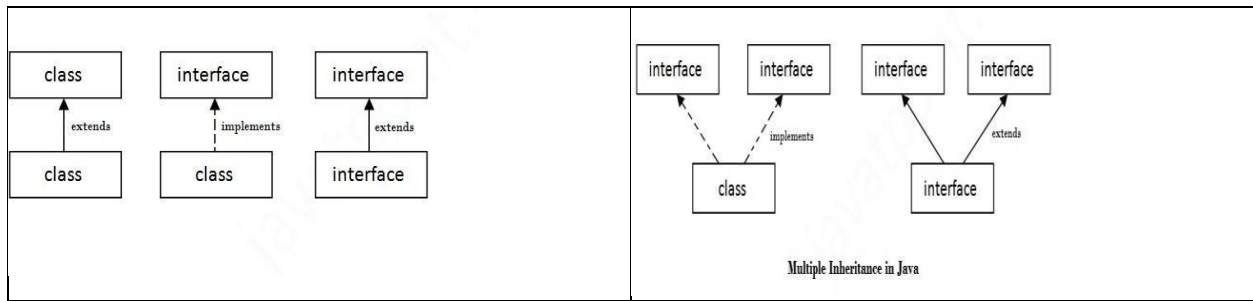
```

class A
{
    int x;
    public A()
    {
        x=0;
    }
    public A(int x)
    {
        this.x=x;
    }
    public int getX()
    {
        return(x+10);
    }
}
class B extends A
{
    int x=20;
    public int getX()
    {
        return(x+10);
    }
    public int getXa()
    {
        return super.x;
    }
    public void show()
    {
        System.out.println(super.getX());
    }
    B(int x)
    {
        super(x);
        System.out.println("Super Parameterized Constructor");
    }
}
public class First
{
    public static void main(String[] args)
    {
        B b = new B(10);
        System.out.println(b.getXa());
        System.out.println(b.getX());
        b.show();
    }
}

```

Topic 04 [inheritance]

Problem Statement: Implement the following relation diagram into java program



Code:

```
class A
{
    A()
    {
        System.out.println("A created");
    }
}

class B extends A
{
    //super(A);
}

public class Topic41 {
    public static void main(String[] args)
    {
        A b = new B();
    }
}

interface A
{
    void show();
}

class B implements A
{
    @Override
    public void show()
    {
        System.out.println("Implemneted from A");
    }
}

public class Topic42 {
    public static void main(String[] args) {
        B a = new B();
        a.show();
    }
}
```

```

interface a
{
    void show();
}
interface b extends a {
    @Override
    default void show()
    {
        System.out.println("extened from a");
    }
}
public class Topic43 {
    public static void main(String[] args) {
        b obj = new b() {};
        obj.show();
    }
}

```

```

interface A{
    void show();
}
interface B{
    void display();
}
class C implements A,B
{
    @Override
    public void show()
    {
        System.out.println("From a");
    }
    @Override
    public void display()
    {
        System.out.println("From b");
    }
}
public class topic44 {
    public static void main(String[] args)
    {
        C c = new C();
        c.show();
        c.display();
    }
}

```

```

interface A
{
    void showA();
}

```

```

}
interface B
{
    void showB();
}
interface C extends A,B
{
    @Override
    default void showA()
    {
        System.out.println("From A");
    }
    @Override
    default void showB()
    {
        System.out.println("From B");
    }
}
public class topic45 {
    public static void main(String[] args)
    {
        C c = new C(){};
        c.showA();
        c.showB();
    }
}

```

Topic 05 [Singleton class]

Problem Statement: Write a Java program to create and operate a singleton class.

```

class Singleton
{
    public static int count=0;
    private static Singleton[] instance= new Singleton[2];
    private Singleton()
    {
        count++;
    }
    public static Singleton getInstance()
    {
        if(count < 2)
        {
            instance[count] = new Singleton();
        }
        return instance[count];
    }
    public void show()
    {
        System.out.println("Singleton instance");
    }
}

```

```

    }
}
public class Topic5
{
    public static void main(String[] args)
    {
        Singleton single1 = Singleton.getInstance();
        Singleton single2 = Singleton.getInstance();

        System.out.println(Singleton.count);
        System.out.println(single2.count);
        single1.show();
    }
}

```

Topic 06 [Java Multithreading]

Problem Statement: Write a program to run three applications i) Word Processor ii) Music Player and iii) Real-time clock simultaneously. Use run() method of Thread class or implements runnable interface.

```

class Wordprocessor extends Thread
{
    public void run()
    {
        for(int i=0; i<10; i++)
        {
            System.out.println("Word processor is running");
        }
    }
}
class Musicplayer extends Thread
{
    public void run()
    {
        for(int i=0; i<10; i++)
        {
            System.out.println("Musicplayer is running");
        }
    }
}
class Realtimelock extends Thread
{
    public void run()
    {
        for(int i=0; i<10; i++)
        {
            System.out.println("Timer is on");
        }
    }
}
public class Topic6 {
    public static void main(String[] args)

```

```

    {
        Wordprocessor word = new Wordprocessor();
        Musicplayer music = new Musicplayer();
        Realtimeclock timer = new Realtimeclock();
        word.start();
        music.start();
        timer.start();
    }
}

```

```

class words implements Runnable

```

```

{
    public void run()
    {
        for(int i=0; i<10; i++)
            System.out.println("Ms Word is running");
    }
}

```

```

class music implements Runnable

```

```

{
    public void run()
    {
        for(int i=0; i<10; i++)
            System.out.println("Music is playing");
    }
}

```

```

}
class timer implements Runnable

```

```

{
    public void run()
    {
        for(int i=0; i<10; i++)
            System.out.println("Time is ticking");
    }
}

```

```

public class Topic6runnable

```

```

{
    public static void main(String[] args)
    {
        words w = new words();
        Thread w1= new Thread(w);
        music m = new music();
        Thread m1 = new Thread(m);
        timer t = new timer();
        Thread t1 = new Thread(t);
        w1.start();
        m1.start();
        t1.start();
    }
}

```

Heaven's Light is our Guide

Rajshahi University of Engineering & Technology

Department of Computer Science & Engineering



Lab Report

Course Code : CSE 1204
Course Name : Object Oriented Programming Sessional
Experiment No : 09
Experiment's Name : Java Swing Programming

Submitted By :		Submitted To :	
Name : Mst. Sadia Akter		Dr. Md. Shahid Uz Zaman	
Roll : 2403033		Professor	
Section : A		Shyla Afroge	
Session : 2024-2025		Associate Professor	
		Department of CSE ,	
		RUET	

Date of Submission :

Module 9

[Advanced Topics: Java Swing Programming]

Topic 1[java swing: Money Changer]

Problem Statement: Write a Money Changer (Dollar to Taka) program in Java with the following components. When user clicks **Convert** button, its shows the output in a label.

Code:

```
public class MoneyChanger {
    public static void main(String[] args) {
        javax.swing.JFrame frame = new javax.swing.JFrame("Money Changer");
        frame.setSize(350, 220);
        frame.setDefaultCloseOperation(javax.swing.JFrame.EXIT_ON_CLOSE);
        frame.setLayout(null);

        javax.swing.JLabel inputLabel = new javax.swing.JLabel("Input $");
        inputLabel.setBounds(60, 40, 80, 25);
        frame.add(inputLabel);

        javax.swing.JTextField dollarField = new javax.swing.JTextField();
        dollarField.setBounds(60, 70, 80, 30);
        frame.add(dollarField);

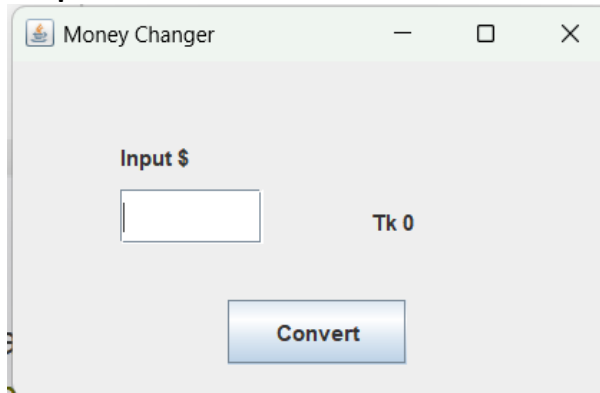
        javax.swing.JLabel resultLabel = new javax.swing.JLabel("Tk 0");
        resultLabel.setBounds(200, 75, 100, 25);
        frame.add(resultLabel);

        javax.swing.JButton convertButton = new javax.swing.JButton("Convert");
        convertButton.setBounds(120, 130, 100, 35);
        frame.add(convertButton);

        convertButton.addActionListener(e -> {
            try {
                double dollar = Double.parseDouble(dollarField.getText());
                double taka = dollar * 110;    // 1 Dollar = 110 Taka
                resultLabel.setText("Tk " + (int)taka);
            } catch (NumberFormatException ex) {
                resultLabel.setText("Invalid Input");
            }
        });

        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
    }
}
```

Output:



Topic 2[java swing: Login Form]

Problem Statement: Write a Java program that accepts username and password of user and then displays whether the user is valid or NOT., It shows the output in a label.

Code:

```
public class LoginForm {  
    public static void main(String[] args) {  
        javax.swing.JFrame frame = new javax.swing.JFrame("Login Form");  
        frame.setSize(360, 250);  
        frame.setDefaultCloseOperation(javax.swing.JFrame.EXIT_ON_CLOSE);  
        frame.setLayout(null);  
  
        javax.swing.JLabel titleLabel = new javax.swing.JLabel("Login Form");  
        titleLabel.setBounds(30, 10, 100, 25);  
        frame.add(titleLabel);  
  
        javax.swing.JLabel userLabel = new javax.swing.JLabel("Username");  
        userLabel.setBounds(40, 60, 80, 25);  
        frame.add(userLabel);  
  
        javax.swing.JTextField userField = new javax.swing.JTextField();  
        userField.setBounds(130, 60, 140, 25);  
        frame.add(userField);  
  
        javax.swing.JLabel passLabel = new javax.swing.JLabel("Password");  
        passLabel.setBounds(40, 100, 80, 25);  
        frame.add(passLabel);  
  
        javax.swing.JPasswordField passField = new javax.swing.JPasswordField();  
        passField.setBounds(130, 100, 140, 25);  
        frame.add(passField);  
  
        javax.swing.JButton submitButton = new javax.swing.JButton("Submit");  
        submitButton.setBounds(120, 145, 90, 35);  
        frame.add(submitButton);  
    }  
}
```



```
javax.swing.JLabel resultLabel = new javax.swing.JLabel("");
resultLabel.setBounds(120, 185, 120, 25);
frame.add(resultLabel);

submitButton.addActionListener(e -> {
    String username = userField.getText();
    String password = new String(passField.getPassword());

    if (username.equals("admin") && password.equals("1234")) {
        resultLabel.setText("Valid User");
    } else {
        resultLabel.setText("Invalid User");
    }
});

frame.setLocationRelativeTo(null);
frame.setVisible(true);
}
}
```

Output: